



Rapport Projet ChaCha20

Conception d'un Système Numérique

Autrice : Nour ZENKRI

Date : 15 Mai 2026

Cours : ISMIN 1A – EI25



Table des matières

Table des tableaux	3
Table des figures	3
1 Introduction	4
1.1 Contexte et objectif	4
1.2 Structure du projet	4
1.3 Rappel de l'algorithme	4
2 Module ARX	5
2.1 Description fonctionnelle	5
2.2 Interface du module	5
2.3 Validation par simulation	5
3 Le module QuarterRound	7
3.1 Description fonctionnelle	7
3.2 Interface du module	7
3.3 Architecture interne	8
3.4 Validation par simulation	8
4 Rondes en colonne et en diagonale	9
4.1 Principe	9
4.2 Interface des modules	9
4.3 Validation par simulation	9
5 Module ChaCha20 Block	10
5.1 Description fonctionnelle	10
5.2 Architecture	10
5.3 Chronogramme et validation	11
6 Machine d'états (FSM)	13
6.1 Rôle et modèle	13
6.2 Description du graphe d'états	13
6.3 Validation par simulation	14
7 Module chacha_top	15
7.1 Architecture générale	15
7.2 Initialisation de l'état	16
7.3 Module XOR : chacha_xor	16
7.4 Validation par simulation	17
8 Conclusion	18

Table des tableaux

- Tabl. 1 : Signaux du module ARX (p. 5)
- Tabl. 2 : Vecteurs de test ARX7 (p. 5)
- Tabl. 3 : Signaux du module QuarterRound (p. 6)
- Tabl. 4 : Vecteurs de test QuarterRound (p. 7)
- Tabl. 5 : Interface des modules Round (p. 8)

Table des figures

- **Fig. 1** : Simulations de l'ARX7 – première et deuxième passes (p. 7)
- **Fig. 2** : Schéma interne du QuarterRound (p. 7)
- **Fig. 3** : Simulation QuarterRound (p. 8)
- **Fig. 4** : Résultats théoriques Round 0 (p. 9)
- **Fig. 5** : Simulations Round colonne et Round diagonale (p. 9)
- **Fig. 6** : Schéma du ChaCha20 Block (p. 10)
- **Fig. 7** : Chronogramme du ChaCha20 Block (p. 11)
- **Fig. 8** : Valeurs intermédiaires attendues (p. 11)
- **Fig. 9** : Valeur du keystream en sortie (p. 12)
- **Fig. 10** : Graphe des états de la FSM (p. 13)
- **Fig. 11** : Résultats du test 1 de la FSM (p. 14)
- **Fig. 12** : Résultats des tests 2 et 3 de la FSM (p. 14)
- **Fig. 13** : Schéma du module `chacha_top` (p. 15)
- **Fig. 14** : Schéma interne du module `chacha_xor` (p. 15)
- **Fig. 15** : Chronogramme de `chacha_top` (p. 16)
- **Fig. 16** : Résultats de la simulation finale (p. 17)

1 Introduction

1.1 Contexte et objectif

ChaCha20 est un algorithme de chiffrement symétrique de type stream cipher, développé par Daniel J. Bernstein. Il est apprécié à la fois pour ses performances élevées et pour son haut niveau de sécurité. Contrairement aux chiffrements par blocs classiques, ChaCha20 produit un flot de données pseudoaléatoires (keystream) qui est ensuite combiné au message en clair à l'aide d'une opération XOR. Cette approche le rend particulièrement efficace pour les applications en temps réel, notamment les communications réseau.

Ce projet a pour objectif de réaliser une implémentation matérielle de ChaCha20 en SystemVerilog. Le développement se fera de manière progressive, en partant de la primitive ARX jusqu'à la conception complète du module de chiffrement. `chacha_top`.

1.2 Structure du projet

Le projet est organisé selon l'arborescence suivante :

Arborescence du projet

```
ChaCha20/  
  LIB/  
    LIB_RTL/      <- Bibliothèques compilées (RTL)  
    LIB_BENCH/    <- Bibliothèques compilées (Testbench)  
  SRC/  
    RTL/          <- Fichiers SystemVerilog des modules  
    BENCH/        <- Fichiers testbench (_tb.sv)
```

Chaque module a un fichier `.sv` dans `SRC/RTL` et un testbench associé dans `SRC/BENCH`. Les règles de nommage sont respectées : suffixe `_i` pour les entrées, `_o` pour les sorties et `_s` pour les signaux internes.

1.3 Rappel de l'algorithme

ChaCha20 opère sur un état interne de 512 bits représenté comme une matrice 4×4 de mots de 32 bits. L'entrée est composée d'une clé K de 256 bits, d'un nonce N de 96 bits et d'un compteur de bloc de 32 bits. L'algorithme effectue ensuite 20 rondes alternant des Column Rounds et des Diagonal Rounds, chacune composée de quatre opérations QuarterRound. Le keystream final est obtenu par addition mot-à-mot de l'état transformé avec l'état initial.

2 Module ARX

2.1 Description fonctionnelle

Le module ARX est la brique élémentaire de ChaCha20. Il enchaîne trois opérations combinatoires sur des mots de 32 bits :

1. **Addition** : $\text{sum} = \text{augend} + \text{addend}$. La retenue est implicitement ignorée en SystemVerilog sur des signaux `logic [31:0]`.
2. **XOR bit à bit** : $\text{xor_result} = \text{sum} \hat{=} \text{xor_i}$.
3. **Rotation circulaire gauche de n bits** :
 $\text{shift} = \{\text{xor_result}[31-n:0], \text{xor_result}[31:32-n]\}$.

La valeur de n change selon l'instance (7, 8, 12 ou 16 bits). Le paramètre **SHIFT** étant la seule chose qui change, il devient assez simple d'implémenter les trois autres variantes ARX7, ARX8, ARX12 et ARX16.

2.2 Interface du module

TABLE 1 – Signaux du module ARX

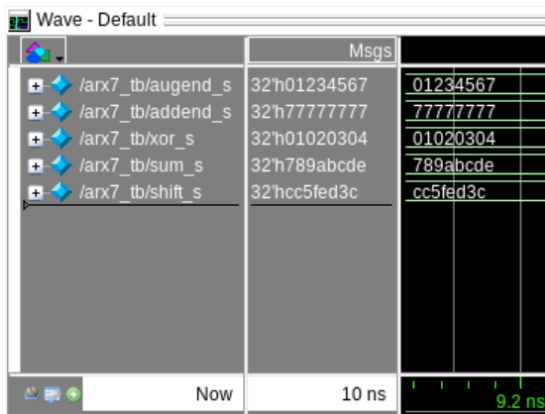
Signal	Largeur	Direction	Description
<code>augend_i</code>	32 bits	Entrée	Premier opérande de l'addition
<code>addend_i</code>	32 bits	Entrée	Deuxième opérande de l'addition
<code>xor_i</code>	32 bits	Entrée	Opérande du XOR (appliqué sur la somme)
<code>sum_o</code>	32 bits	Sortie	Résultat de l'addition <code>augend + addend</code>
<code>shift_o</code>	32 bits	Sortie	Résultat après XOR puis rotation gauche de n bits

2.3 Validation par simulation

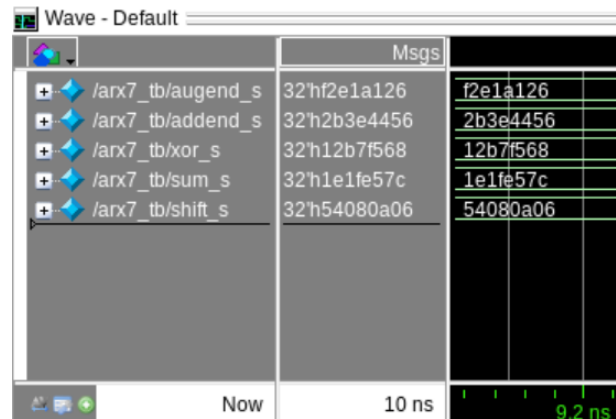
Pour valider le bon fonctionnement, j'utilise en priorité l'instance ARX7 car les valeurs de référence sont données dans l'énoncé. Les deux vecteurs de test sont les suivants :

TABLE 2 – Vecteurs de test ARX7 issus de l'énoncé

Signal	Vecteur 1	Vecteur 2
<code>augend_i</code>	0x01234567	0xF2E1A126
<code>addend_i</code>	0x77777777	0x2B3E4456
<code>xor_i</code>	0x01020304	0x12B7F568
<code>sum_o</code>	0x789ABCDE	0x1E1FE57C
<code>shift_o</code>	0xCC5FED3C	0x54080A06



(a) Première simulation (vecteur 1)



(b) Deuxième simulation (vecteur 2)

FIGURE 1 – Simulations de l'ARX7 – les sorties correspondent aux valeurs théoriques

Les deux simulations confirment que les sorties `sum_o` et `shift_o` correspondent aux valeurs de référence.

Remarque d'implémentation

J'aurais pu n'utiliser qu'un seul module ARX avec une entrée supplémentaire permettant de sélectionner la valeur du SHIFT. Je ne me suis rendu compte de cette possibilité qu'au moment de rédiger ce rapport. J'ai sûrement préféré séparer les modules, car je souhaitais garder une conception simple sachant qu'il s'agissait du premier module que j'implémentais.

3 Le module QuarterRound

3.1 Description fonctionnelle

Le QuarterRound est l'opération élémentaire de ChaCha20. Il prend en entrée quatre mots de 32 bits (a, b, c, d) et leur applique successivement quatre opérations ARX avec des rotations différentes (16, 12, 8 et 7 bits) :

$$\begin{aligned} a &= a + b; & d &= (d \oplus a)16 \\ c &= c + d; & b &= (b \oplus c)12 \\ a &= a + b; & d &= (d \oplus a)8 \\ c &= c + d; & b &= (b \oplus c)7 \end{aligned}$$

Ce schéma crée un effet avalanche : une modification d'un seul bit en entrée se propage rapidement à l'ensemble des 128 bits de sortie, ce qui assure les propriétés cryptographiques de ChaCha20.

3.2 Interface du module

TABLE 3 – Signaux du module QuarterRound

Signal	Largeur	Direction	Description
a_i	32 bits	Entrée	Premier mot d'entrée
b_i	32 bits	Entrée	Deuxième mot d'entrée
c_i	32 bits	Entrée	Troisième mot d'entrée
d_i	32 bits	Entrée	Quatrième mot d'entrée
a_o	32 bits	Sortie	Premier mot transformé
b_o	32 bits	Sortie	Deuxième mot transformé
c_o	32 bits	Sortie	Troisième mot transformé
d_o	32 bits	Sortie	Quatrième mot transformé

3.3 Architecture interne

Le QuarterRound instancie quatre modules ARX chaînés. Les sorties de chaque ARX alimentent les entrées du suivant, ce qui forme un chemin de données purement combinatoire. Le schéma est donné en Figure 2.

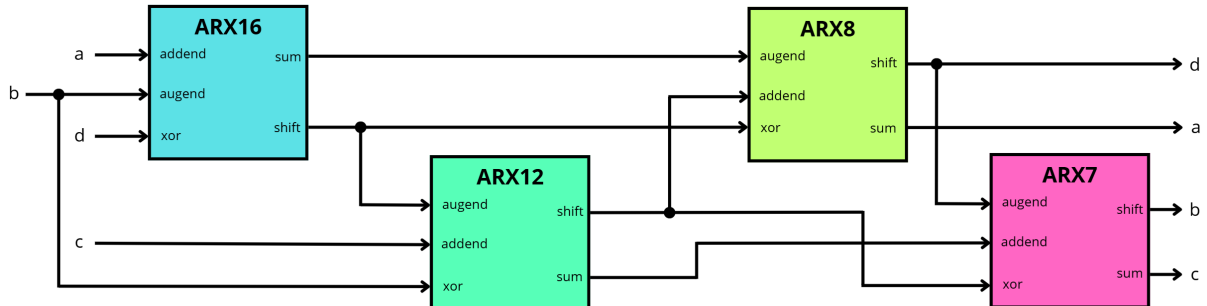


FIGURE 2 – Schéma interne du QuarterRound : quatre modules ARX chaînés

Choix d'implémentation

Le QuarterRound est combinatoire pur, sans registres intermédiaires. Cela permet d'exécuter une opération QuarterRound complète en un seul cycle d'horloge au niveau du `round_colonne` et `round_diagonale`.

3.4 Validation par simulation

TABLE 4 – Vecteurs de test QuarterRound

Signal	Entrée	Sortie attendue
a	0x11111111	0xEA2A92F4
b	0x01020304	0xCB1CF8CE
c	0x9B8D6F43	0x4581472E
d	0x01234567	0x5881C4BB

Le chronogramme confirme que `a_o`, `b_o`, `c_o` et `d_o` correspondent exactement aux valeurs théoriques, ce qui valide l'enchaînement des quatre modules ARX et le câblage combinatoire.

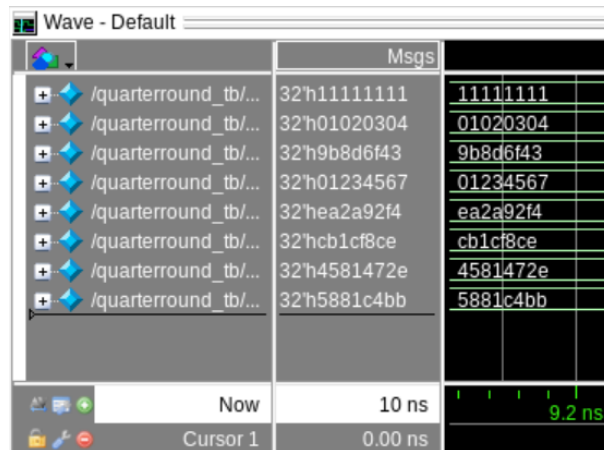


FIGURE 3 – Simulation du QuarterRound : les quatre sorties correspondent aux valeurs de référence

4 Rondes en colonne et en diagonale

4.1 Principe

L'état interne de ChaCha20 est une matrice 4×4 de mots de 32 bits. Une ronde consiste à appliquer simultanément quatre QuarterRound sur des quadruplets de cette matrice. L'alternance colonne/diagonale assure une bonne diffusion de l'information dans l'état.

- **Ronde en colonne** : les QuarterRound opèrent sur les colonnes.
 $QR(S,0,4,8,12)$ $QR(S,1,5,9,13)$ $QR(S,2,6,10,14)$ $QR(S,3,7,11,15)$
- **Ronde en diagonale** : les QuarterRound opèrent sur les diagonales décalées.
 $QR(S,0,5,10,15)$ $QR(S,1,6,11,12)$ $QR(S,2,7,8,13)$ $QR(S,3,4,9,14)$

4.2 Interface des modules

TABLE 5 – Interface commune aux modules Round colonne et Round diagonale

Signal	Largeur	Direction	Description
mat_i	16×32 bits	Entrée	Matrice d'état en entrée (512 bits)
mat_o	16×32 bits	Sortie	Matrice d'état transformée en sortie (512 bits)

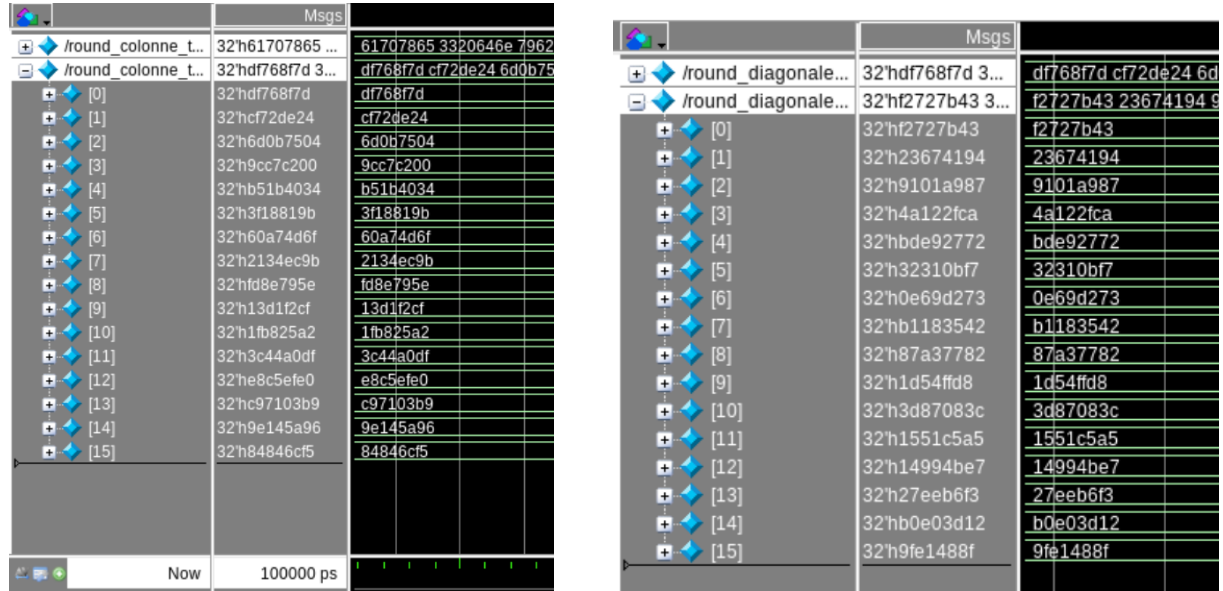
4.3 Validation par simulation

Les deux modulesinstancient quatre QuarterRound en parallèle. La seule différence est le câblage des entrées : colonnes pour le premier, diagonales décalées pour le second. Cette parallélisation garantit qu'une ronde entière s'exécute en un seul cycle d'horloge.

Les chronogrammes confirment que les sorties `mat_o` des deux modules correspondent aux matrices de référence de l'énoncé pour le Round 0.

Ronde #0	:	df768f7d	cf72de24	6d0b7504	9cc7c200	f2727b43	23674194	9101a987	4a122fca
		b51b4034	3f18819b	60a74d6f	2134ec9b	bde92772	32310bf7	0e69d273	b1183542
		fd8e795e	13d1f2cf	1fb825a2	3c44a0df	87a37782	1d54ffd8	3d87083c	1551c5a5
		e8c5efe0	c97103b9	9e145a96	84846cf5	14994be7	27eeb6f3	b0e03d12	9fe1488f

FIGURE 4 – Résultats théoriques après le Round 0 : ronde colonne (gauche) et ronde diagonale (droite)



(a) Simulation – Ronde colonne

(b) Simulation – Ronde diagonale

FIGURE 5 – Validation des deux modules – correspondance avec les valeurs théoriques

5 Module ChaCha20 Block

5.1 Description fonctionnelle

Le module `chacha_bloc` exécute les 20 rondes de l'algorithme (10 rondes en colonne alternées avec 10 en diagonale) à partir d'un état initial, et produit le *keystream* par addition finale avec cet état initial.

Plutôt que de câbler 20 modules Round en série, ce qui serait très coûteux en matériel, j'ai opté pour une architecture itérative avec registre de rebouclage : deux modules (Round colonne suivi de Round diagonale) est réexécuté 10 fois grâce à un registre qui mémorise l'état courant entre chaque itération.

5.2 Architecture

L'architecture comporte les éléments suivants :

- Un **multiplexeur 2 :1** commandé par `init_i` : il sélectionne la matrice initiale au premier cycle, puis la sortie du registre lors des cycles suivants.
- Un **Round colonne** suivi d'un **Round diagonale** câblés en série et purement combinatoires : ensemble ils forment une ronde complète de ChaCha20 en un seul cycle d'horloge.
- Un **registre d'état de 512 bits**, synchrone sur le front montant, dont l'écriture

est conditionnée par `enable_i` issu de la FSM. Il mémorise l'état intermédiaire après chaque ronde.

- Un **additionneur final** de 16 mots de 32 bits qui additionne l'état après les 10 rondes avec l'état initial pour produire le *keystream* sur `mat_o`.

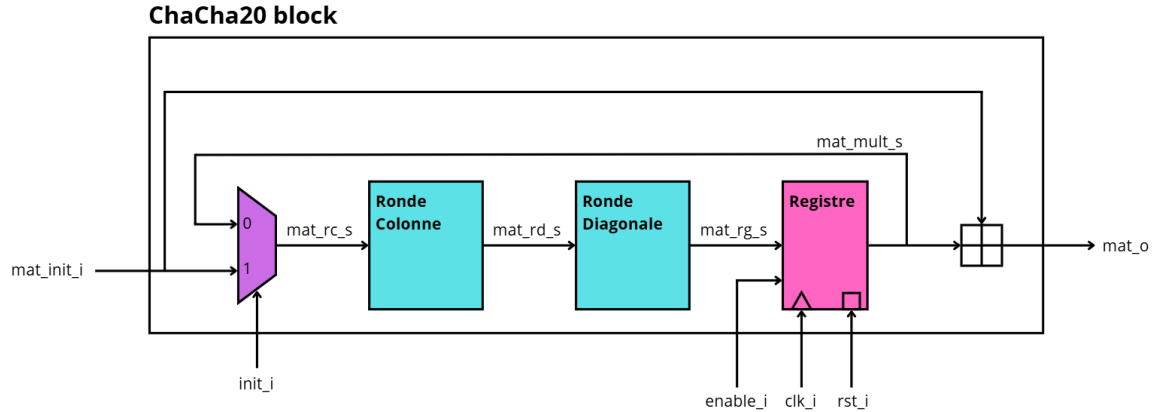


FIGURE 6 – Architecture du module ChaCha20 Block avec registre de rebouclage

Choix d'implémentation

J'aurais pu n'utiliser qu'un seul module `round_colonne` avec un multiplexeur pour alterner colonne et diagonale, ce qui aurait réduit le coût matériel. J'ai préféré garder les deux modules séparés car c'est plus simple à déboguer et le gain matériel n'était pas si important dans le cadre de ce projet.

5.3 Chronogramme et validation

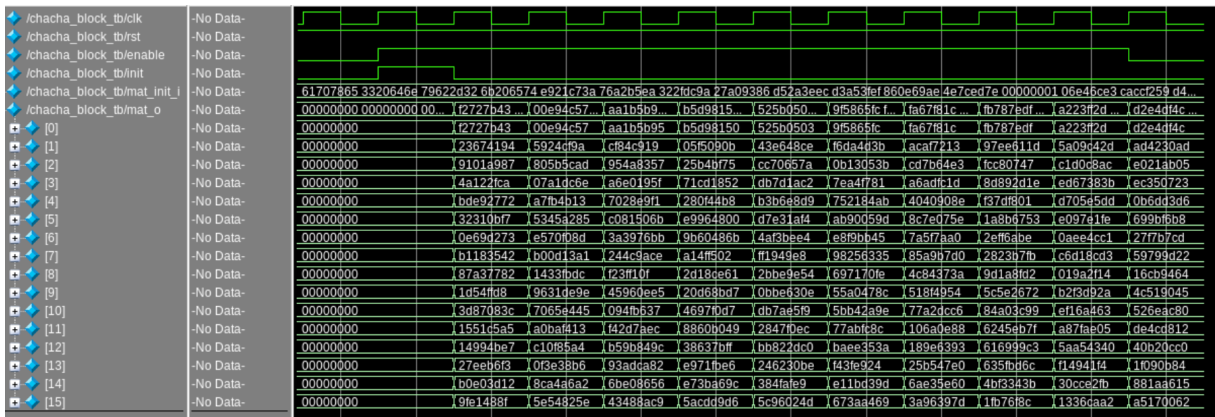


FIGURE 7 – Chronogramme du module ChaCha20 Block (sans l'addition finale)

L'analyse du chronogramme permet de vérifier le comportement cycle par cycle :

- **Cycle 1** : `init_i` = 1, le multiplexeur sélectionne la matrice initiale. Le registre est activé (`enable` = 1) et enregistre le résultat de la première ronde.
- **Cycles 2 à 10** : `init_i` = 0, le registre reboucle sa sortie vers l'entrée du multiplexeur. À chaque front montant, l'état est mis à jour avec le résultat de la ronde suivante.

- **Après 10 cycles** : toutes les rondes sont calculées. L'additionneur final produit le *keystream* sur `mat_o`.

```

-----
Ronde #6      : fe2dce69 6bf21e78 7c5bc8e2 f79ef9a0 fa67f81c acaf7213 cd7b64e3 a6adfc1d
                61ff3a08 318008eb f01ea31e 639037ac 4040908e 8c7e075e 7a5f7aa0 85a9b7d0
                f9701db6 1fe4cbb1 b5af3524 b813be85 4c84373a 518f4954 77a2dcc6 106a0e88
                b060fe21 0e482dd0 46ddf56f 41885009 189e6393 25b547e0 6ae35e60 3a96397d
-----
Ronde #7      : 17487029 1f589b36 df10d7c0 658145e2 fb787edf 97ee611d fcc80747 8d892d1e
                c84f09ba c19ae45f 5a7db03d b26c66ca f37df801 1a8b6753 2eff6abe 2823b7fb
                a90f796c 59a8140d ed81035d ac4d4938 9d1a8fd2 5c5e2672 84a03c99 6245eb7f
                71521ffc c987ae21 f3faf95e 115323ef 616999c3 635fbd6c 4bf3343b 1fb76f8c
-----

```

13/17

Conception d'un Système Numérique
ISMIN 1A – EI25

CHIFFREMENT CHACHA20

```

-----
Ronde #8      : 2d6d7ed4 cb86852f 4b2eb047 37d30168 a223ff2d 5a09c42d c1d0c8ac ed67383b
                ac3663f2 61def4b9 6418cc62 d08a8c58 d705e5dd e097e1fe 0aee4cc1 c6d18cd3
                db2f6b33 6bcf0156 dbaf0fda 33870957 019a2f14 b2f3d92a ef16a463 a87fae05
                4ef14bc2 9a5409be 10d0730d 46ab73bd 5aa54340 f14941f4 30cce2fb 1336caa2
-----
Ronde #9      : 94a1bc18 7ef06dcf 0b809431 91d4ccfd d2e4df4c ad4230ad e021ab05 ec350723
                7a18ee44 1b8587c7 ea4e8b61 8227af76 0b6dd3d6 699bf6b8 27f7b7cd 59799d22
                9383e6d2 ca79ccab fd15e3d2 309848b1 16cb9464 4c519045 526eac80 de4cd812
                eb9f9432 2fa62799 166842fc 786bf39e 40b20cc0 1f090b84 881aa615 a5170062
*****

```

FIGURE 8 – États intermédiaires attendus par l'énoncé, utilisés pour valider chaque ronde

Les valeurs intermédiaires après chaque ronde correspondent aux matrices de référence de l'énoncé, ce qui valide le module `chacha_bloc`. Une fois l'addition avec l'état initial intégrée, le *keystream* est disponible sur `mat_o`. Sa valeur est confirmée par la Figure 9.

```

VSIM 2> run 600ns
# RESULTAT
# mat_o[0] = 345557b1
# mat_o[1] = e062951b
# mat_o[2] = 5983d837
# mat_o[3] = 57556c97
# mat_o[4] = f48f9b10
# mat_o[5] = e03eaca2
# mat_o[6] = 5a279467
# mat_o[7] = 811a30a8
# mat_o[8] = ebf5d350
# mat_o[9] = 1ff6d034
# mat_o[10] = d87d162e
# mat_o[11] = 2cc9c590
# mat_o[12] = 40b20cc1
# mat_o[13] = 25ed7867
# mat_o[14] = 52e7986e
# mat_o[15] = 79c3921b
# ** Note: $stop : ./SRC/BENCH/chacha_bloc_tb.sv(79)
# Time: 145 ns Iteration: 0 Instance: /chacha_bloc_tb
# Break in Module chacha_bloc_tb at ./SRC/BENCH/chacha_blo

```

FIGURE 9 – Valeur du *keystream* affichée en simulation, correspondant à la valeur de référence de l'énoncé

6 Machine d'états (FSM)

6.1 Rôle et modèle

La FSM (*Finite State Machine*) est le module de contrôle central du chiffrement. Elle orchestre le `chacha_bloc`, le compteur de rondes et les signaux de validité. Le modèle adopté est une FSM de Moore : les sorties ne dépendent que de l'état courant, et non des états futurs.

6.2 Description du graphe d'états

La FSM se décompose en deux grandes phases : une phase de calcul du *keystream* et une phase de chiffrement du message.

Phase 1 : Calcul du *keystream*

La FSM démarre en `IDLE` et attend `start_i`. Une fois reçu, elle passe en `INIT` pour charger l'état initial et remettre le compteur à zéro, puis enchaîne immédiatement sur `ROUND`. À chaque cycle d'horloge, une ronde complète (colonne + diagonale) est calculée et mémorisée. Le compteur s'incrémente à chaque cycle. La condition de sortie est fixée à `counter_i = 8` : lors du débogage, j'avais d'abord mis `counter_i = 9`, ce qui produisait 11 rondes au lieu de 10. En analysant la simulation, j'ai réalisé que l'incréméntation du compteur se faisait dans l'état `ROUND` qui s'exécute réellement après un cycle d'horloge, d'où le décalage. Lorsque `counter_i = 8`, les 10 rondes sont disponibles après le front d'horloge suivant. La FSM transite alors vers `KEYSTREAM` qui signale, via `end_o = 1`, que le *keystream* est prêt.

Phase 2 : Chiffrement du message

Une fois le *keystream* calculé, la FSM entre en `WAIT` et attend qu'un bloc de texte clair soit disponible (`data_valid_i = 1`). Elle passe ensuite en `CIPHER` pour effectuer le XOR, puis en `MEM` pour mémoriser le résultat. Ce cycle `WAIT` → `CIPHER` → `MEM` se répète pour chacun des trois blocs. Lorsque le troisième bloc est traité (`counter_i = 3`), la FSM transite vers `DONE` puis retourne en `IDLE`.

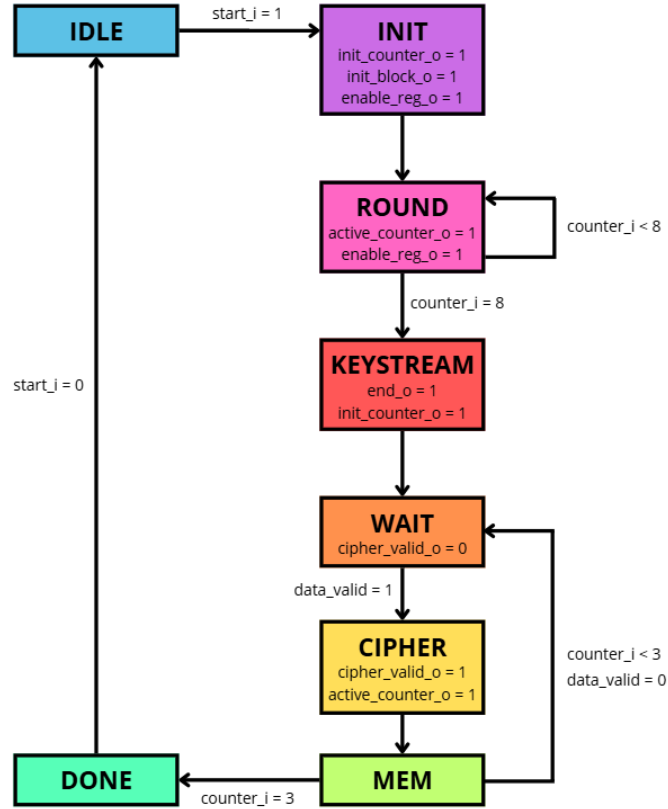


FIGURE 10 – Graphe des états de la FSM de contrôle (modèle Moore)

6.3 Validation par simulation

Un testbench dédié `chacha_fsm_tb.sv` a été développé. Il instancie le module `chacha_fsm` et simule un compteur externe qui reproduit le comportement réel : le compteur s'incrémente lorsque `active_counter_o = 1` et se réinitialise lorsque `init_counter_o = 1`, ce qui permet de tester la FSM sans intégrer le `chacha_bloc` complet.

Trois scénarios de test sont couverts :

- **Test 1 | Fonctionnement nominal avec délai** : après activation de `start_i`, la FSM parcourt les états `INIT` → `ROUND` (10 itérations) → `KEYSTREAM` → `WAIT`. La détection du front montant de `end_o` confirme que le keystream est prêt. Les trois blocs sont ensuite envoyés avec un délai de deux cycles entre chaque, ce qui vérifie que la FSM attend correctement en `WAIT`.
- **Test 2 | data_valid immédiat** : les trois blocs sont envoyés sans délai après `end_o`. Ce test vérifie que la FSM transite directement de `WAIT` vers `CIPHER` dès la première donnée valide, et enchaîne correctement les états `CIPHER` → `MEM` pour les trois blocs avant d'atteindre `DONE`.
- **Test 3 | Reset en cours de calcul** : un reset asynchrone (`resetb_i = 0`) est appliqué pendant `ROUND`. Ce test vérifie que la FSM retourne immédiatement en `IDLE` et que le compteur est remis à zéro.

Un `$monitor` continu affiche à chaque changement le nom de l'état courant, la valeur du compteur et l'ensemble des signaux de contrôle, ce qui permet de confirmer la cohérence de chaque transition avec le graphe d'états de la Figure 10.


```

VSIM 2> run 600ns
# t=0 | state=IDLE | counter=0 | end=0 | cipher_valid=0 | init=0 | active=0 | enable=0 | init_block=0
# Test 1
# t=45000 | state=INIT | counter=0 | end=0 | cipher_valid=0 | init=1 | active=0 | enable=1 | init_block=1
# t=55000 | state=ROUND | counter=0 | end=0 | cipher_valid=0 | init=0 | active=1 | enable=1 | init_block=0
# t=65000 | state=ROUND | counter=1 | end=0 | cipher_valid=0 | init=0 | active=1 | enable=1 | init_block=0
# t=75000 | state=ROUND | counter=2 | end=0 | cipher_valid=0 | init=0 | active=1 | enable=1 | init_block=0
# t=85000 | state=ROUND | counter=3 | end=0 | cipher_valid=0 | init=0 | active=1 | enable=1 | init_block=0
# t=95000 | state=ROUND | counter=4 | end=0 | cipher_valid=0 | init=0 | active=1 | enable=1 | init_block=0
# t=105000 | state=ROUND | counter=5 | end=0 | cipher_valid=0 | init=0 | active=1 | enable=1 | init_block=0
# t=115000 | state=ROUND | counter=6 | end=0 | cipher_valid=0 | init=0 | active=1 | enable=1 | init_block=0
# t=125000 | state=ROUND | counter=7 | end=0 | cipher_valid=0 | init=0 | active=1 | enable=1 | init_block=0
# t=135000 | state=ROUND | counter=8 | end=0 | cipher_valid=0 | init=0 | active=1 | enable=1 | init_block=0
# t=145000 : keystream pret, counter=9
# t=145000 | state=KEYSTREAM | counter=9 | end=1 | cipher_valid=0 | init=1 | active=0 | enable=0 | init_block=0
# t=155000 | state=WAIT | counter=0 | end=0 | cipher_valid=0 | init=0 | active=0 | enable=0 | init_block=0
# t=175000 | state=CIPHER | counter=0 | end=0 | cipher_valid=1 | init=0 | active=1 | enable=0 | init_block=0
# t=185000 | state=MEM | counter=1 | end=0 | cipher_valid=0 | init=0 | active=0 | enable=0 | init_block=0
# t=186000 : P1 chiffre, counter=1
# t=195000 | state=WAIT | counter=1 | end=0 | cipher_valid=0 | init=0 | active=0 | enable=0 | init_block=0
# t=215000 | state=CIPHER | counter=1 | end=0 | cipher_valid=1 | init=0 | active=1 | enable=0 | init_block=0
# t=225000 | state=MEM | counter=2 | end=0 | cipher_valid=0 | init=0 | active=0 | enable=0 | init_block=0
# t=226000 : P2 chiffre, counter=2
# t=235000 | state=WAIT | counter=2 | end=0 | cipher_valid=0 | init=0 | active=0 | enable=0 | init_block=0
# t=255000 | state=CIPHER | counter=2 | end=0 | cipher_valid=1 | init=0 | active=1 | enable=0 | init_block=0
# t=265000 | state=MEM | counter=3 | end=0 | cipher_valid=0 | init=0 | active=0 | enable=0 | init_block=0
# t=266000 : P3 chiffre, counter=3
# t=275000 | state=DONE | counter=3 | end=0 | cipher_valid=0 | init=0 | active=0 | enable=0 | init_block=0
# t=285000 | state=IDLE | counter=3 | end=0 | cipher_valid=0 | init=0 | active=0 | enable=0 | init_block=0

```

FIGURE 11 – Résultats en console du test 1 de la FSM

```

# Test 2 : data_valid immediat
# t=305000 | state=INIT | counter=3 | end=0 | cipher_valid=0 | init=1 | active=0 | enable=1 | init_block=1
# t=315000 | state=ROUND | counter=0 | end=0 | cipher_valid=0 | init=0 | active=1 | enable=1 | init_block=0
# t=325000 | state=ROUND | counter=1 | end=0 | cipher_valid=0 | init=0 | active=1 | enable=1 | init_block=0
# t=335000 | state=ROUND | counter=2 | end=0 | cipher_valid=0 | init=0 | active=1 | enable=1 | init_block=0
# t=345000 | state=ROUND | counter=3 | end=0 | cipher_valid=0 | init=0 | active=1 | enable=1 | init_block=0
# t=355000 | state=ROUND | counter=4 | end=0 | cipher_valid=0 | init=0 | active=1 | enable=1 | init_block=0
# t=365000 | state=ROUND | counter=5 | end=0 | cipher_valid=0 | init=0 | active=1 | enable=1 | init_block=0
# t=375000 | state=ROUND | counter=6 | end=0 | cipher_valid=0 | init=0 | active=1 | enable=1 | init_block=0
# t=385000 | state=ROUND | counter=7 | end=0 | cipher_valid=0 | init=0 | active=1 | enable=1 | init_block=0
# t=395000 | state=ROUND | counter=8 | end=0 | cipher_valid=0 | init=0 | active=1 | enable=1 | init_block=0
# t=405000 : keystream pret, counter=9
# t=405000 | state=KEYSTREAM | counter=9 | end=1 | cipher_valid=0 | init=1 | active=0 | enable=0 | init_block=0
# t=415000 | state=WAIT | counter=0 | end=0 | cipher_valid=0 | init=0 | active=0 | enable=0 | init_block=0
# t=425000 | state=CIPHER | counter=0 | end=0 | cipher_valid=1 | init=0 | active=1 | enable=0 | init_block=0
# t=426000 : P1 chiffre, counter=0
# t=435000 | state=MEM | counter=1 | end=0 | cipher_valid=0 | init=0 | active=0 | enable=0 | init_block=0
# t=446000 : P2 chiffre, counter=1
# t=466000 : P3 chiffre, counter=1
# t=475000 | state=WAIT | counter=1 | end=0 | cipher_valid=0 | init=0 | active=0 | enable=0 | init_block=0
# Test 3 : reset en cours
# t=536000 | state=IDLE | counter=0 | end=0 | cipher_valid=0 | init=0 | active=0 | enable=0 | init_block=0
# t=546000 : reset -> retour IDLE, counter=0
# Fin des tests
** Note: $finish : ./SRC/BENCH/chacha_fsm_tb.sv(126)
# Time: 576 ns Iteration: 0 Instance: /chacha_fsm_tb

```

FIGURE 12 – Résultats en console des tests 2 et 3 de la FSM

7 Module chacha_top

7.1 Architecture générale

Le module `chacha_top` est le module d'intégration finale. Il connecte quatre sous-modules : la FSM, le Compteur, le ChaCha20 Block et le module XOR, comme illustré en Figure 13.

La FSM orchestre l'ensemble du système en pilotant le Compteur et le ChaCha20 Block via les signaux de contrôle `init_block_o`, `enable_reg_o`, `init_counter_o` et `active_counter_o`. Le Compteur renvoie `count_o` à la FSM via `counter_i` pour contrôler les transitions d'états. Le ChaCha20 Block reçoit la matrice d'initialisation `mat_init_i` construite dans `chacha_top` à partir de la constante, de la clé `key_i` et du nonce `nonce_i`, et produit le keystream sur `mat_o`. Ce keystream est transmis au module XOR qui produit `cipher_o`.

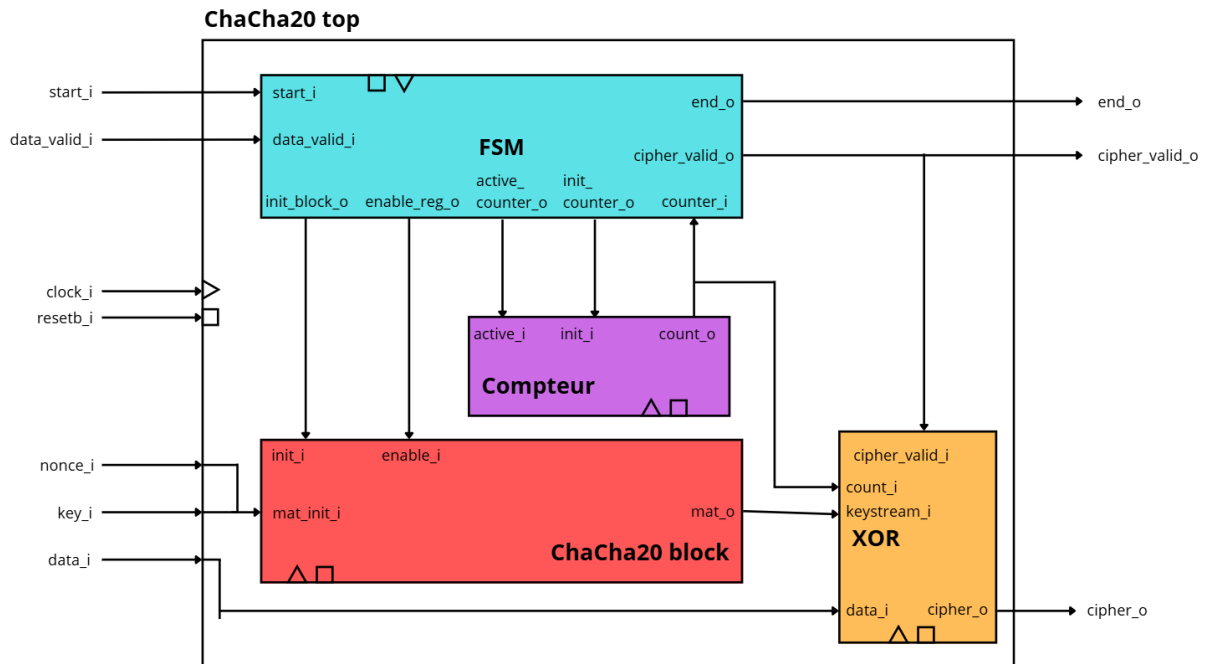


FIGURE 13 – Schéma du module chacha_top

7.2 Initialisation de l'état

L'état initial est construit dans `chacha_top` par des assignations combinatoires (`assign`). La première ligne reçoit la constante ChaCha20 (0x61707865, 0x3320646e, 0x79622d32, 0x6b206574), les deux lignes suivantes reçoivent les huit mots de 32 bits de la clé, et la dernière ligne reçoit le compteur de bloc fixé à 0x00000001 suivi des trois mots du nonce.

7.3 Module XOR : chacha_xor

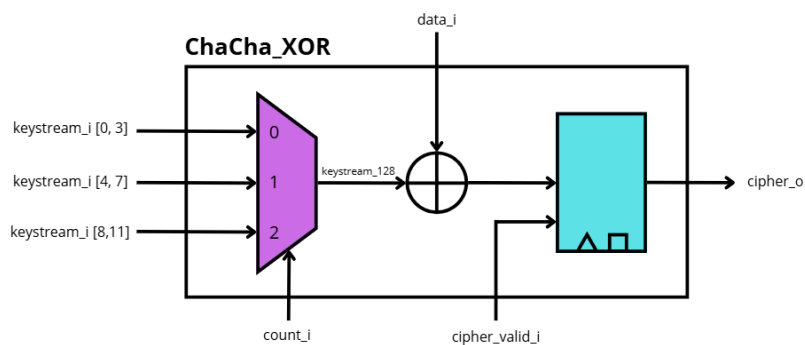


FIGURE 14 – Schéma interne du module chacha_xor

Le module `xor_cipher` sélectionne la bonne tranche du *keystoream* et la combine par XOR avec le bloc de texte clair courant. Son architecture se compose de trois éléments :

- Un **multiplexeur 3 :1** qui sélectionne parmi les trois tranches de 128 bits du *keystoream* – `keystream_i[0:3]`, `keystream_i[4:7]` et `keystream_i[8:11]` – en fonction de `count_i`. Le premier bloc est XORé avec les mots 0 à 3, le deuxième avec les mots 4 à 7, et le troisième avec les mots 8 à 11.
- Un **opérateur XOR** de 128 bits qui combine la tranche sélectionnée avec `data_i`.

- Un **registre de sortie** synchrone activé par `cipher_valid_i`, qui mémorise le résultat du XOR et le présente sur `cipher_o`. Dans ma première implémentation, ce registre n'était pas présent, le XOR était directement combinatoire sur `cipher_o`. En pratique cela peut fonctionner correctement, mais en relisant le sujet et en utilisant le testbench du professeur, j'ai réalisé que l'énoncé imposait ce registre via le signal `cipher_valid_i` utilisé comme `enable`. Je l'ai donc ajouté pour me conformer à la spécification.

7.4 Validation par simulation

La validation finale a été réalisée avec le testbench du professeur `chacha_top_prof_tb.sv`. Ce testbench vérifie automatiquement la conformité du texte chiffré en calculant un XOR bit à bit entre le résultat obtenu et la valeur de référence, et conclut par `Test PASSED` ou `Test FAILED`.

Pour le bon fonctionnement du `test_bench`, j'ai dû ajouter un cycle d'horloge supplémentaire après chaque attente de `cipher_valid_o` avant de lire `cipher_o`. En observant le chronogramme (figure 15), on voit que `cipher_valid_o` passe à 1 sur un front montant, et que `cipher_o` est lisible un cycle plus tard. Cependant, le testbench du professeur lis les sorties sur les fronts descendants, en rajoutant un cycle d'horloge on garantie une lecture stable un cycle et demi après l'activation de `cipher_valid_o`.

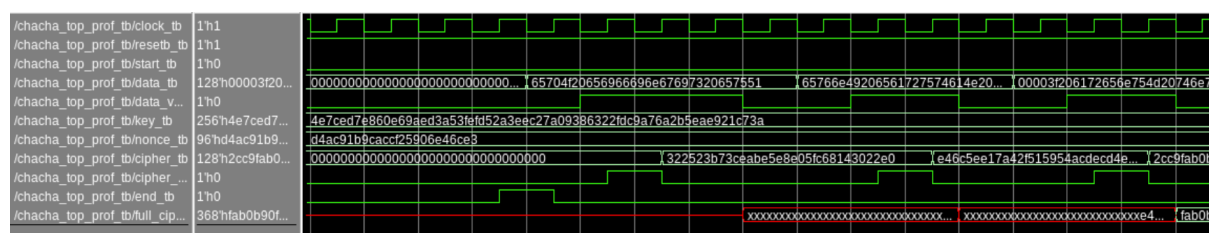


FIGURE 15 – Chronogramme de `chacha_top`

Après ces ajustements, le déroulement est le suivant : après un reset de 20 cycles, la clé et le nonce sont chargés et `start_i` est activé. Le testbench attend `end_o`, puis envoie les trois blocs du message en clair. Les blocs chiffrés sont concaténés dans `full_cipher_tb` de 368 bits pour reconstituer les 46 octets du message complet, les deux octets de poids fort du dernier bloc étant ignorés car ils correspondent au *padding*.

Les résultats attendus sont :

Résultats de référence (énoncé)

Bloc 1 : 322523b73ceabe5e8e05fc68143022e0
 Bloc 2 : e46c5ee17a42f515954acdec4eeef62
 Bloc 3 : 2cc9fab0b90f73406abbf0408580ba3e

La correspondance entre les valeurs simulées et les valeurs de référence, confirmée par `Test PASSED` et un XOR résultant nul, valide le bon fonctionnement de l'intégration complète.

[illegible]

FIGURE 16 – Résultats en console de la simulation avec le testbench du professeur

8 Conclusion

Ce projet a permis d’implémenter en SystemVerilog l’intégralité de l’algorithme ChaCha20, depuis la primitive ARX jusqu’au module `chacha_top`. La démarche hiérarchique – valider chaque module indépendamment avant de l’intégrer s’est révélée efficace pour isoler rapidement les erreurs.

Les points clés de l'implémentation sont :

- Un module ARX paramétrable (paramètre shift) réutilisé pour les quatre variantes ARX7, ARX8, ARX12 et ARX16.
- Un QuarterRound entièrement combinatoire, composé de quatre instances ARX chaînées.
- Une architecture itérative du ChaCha Block avec registre de rebouclage, permettant d'exécuter les 10 rondes en 10 cycles tout en limitant le coût matériel.
- Une FSM de type Moore dont le compteur est partagé entre la phase de calcul des rondes et la phase de chiffrement, ce qui simplifie l'architecture globale.